HDOC Day-46

Bubble Sort and First/Last Occurrence Using Pointers

Problem Statement

Read a variable-length integer array, sort it using Bubble_sort() with pointer-based access, then use occurrence() to read a target and print its first and last occurrences in the sorted array. Use zero-based indices; print -1, -1 if absent.

Algorithm

1. Start and read a positive array size; reject zero and negative sizes.
2. Declare a variable-length integer array nums[size] and read all its elements.
3. Pass nums and size to Bubble_sort() using the array address.
4. In Bubble_sort(), compare adjacent elements through pointers and swap if out of ascending order.
5. Display the sorted array.
6. Pass the sorted array and size to occurrence().
7. Read the target inside occurrence(); scan the sorted array to record first and last matching zero-based indices.
8. If target is absent, print -1, -1; otherwise display target and both indices. Stop.

Test Case Table

TC	Size	Elements	Target	Sorted array	Expected result
1	6	8 5 10 7 8 7	8	5 7 7 8 8 10	First: 8 at index 3; Last: 8 at index 4
2	5	5 7 7 8 10	8	5 7 7 8 10	First: 8 at index 3; Last: 8 at index 3
3	6	5 7 7 8 8 10	6	5 7 7 8 8 10	-1, -1
4	5	4 4 4 4 4	4	4 4 4 4 4	First: 4 at index 0; Last: 4 at index 4
5	1	9	9	9	First: 9 at index 0; Last: 9 at index 0
6	5	-2 0 -2 3 0	-2	-2 -2 0 0 3	First: -2 at index 0; Last: -2 at index 1
7	0 then 3	3 1 2	2	1 2 3	Reject size 0, then First: 2 at index 1; Last: 2 at index 1
8	4	9 1 5 3	9	1 3 5 9	First: 9 at index 3; Last: 9 at index 3

Note: All positions are zero-based indices. For Test Case 7, first enter size 0, then enter size 3 when asked again.

C Program

```
#include <stdio.h>
```

```c
#include <stdlib.h>

void Bubble_sort(int *nums, int size)
{
    int i, j, temp;
    for (i = 0; i < size - 1; i++) {
        for (j = 0; j < size - 1 - i; j++) {
            if (*(nums + j) > *(nums + j + 1)) {
                temp = *(nums + j);
                *(nums + j) = *(nums + j + 1);
                *(nums + j + 1) = temp;
            }
        }
    }
}

void occurrence(const int *nums, int size)
{
    int target, first = -1, last = -1, i;
    printf("Enter target: ");
    if (scanf("%d", &target) != 1) {
        printf("Invalid target input.\n");
        return;
    }
    for (i = 0; i < size; i++) {
        if (*(nums + i) == target) {
            if (first == -1) first = i;
            last = i;
        }
    }
    if (first == -1) {
        printf("-1, -1\n");
    } else {
        printf("First occurrence: %d at index %d\n", *(nums + first), first);
        printf("Last occurrence: %d at index %d\n", *(nums + last), last);
    }
}

int main(void)
{
    int size, i;
    do {
        printf("Enter array size (positive): ");
        if (scanf("%d", &size) != 1) {
            printf("Invalid size input.\n");
            return 1;
        }
        if (size <= 0) printf("Size must be positive.\n");
    } while (size <= 0);

    /* Variable-length integer array, as required. */
    int nums[size];
    printf("Enter %d array elements:\n", size);
    for (i = 0; i < size; i++) {
        if (scanf("%d", &nums[i]) != 1) {
            printf("Invalid array element.\n");
            return 1;
        }
    }

    Bubble_sort(nums, size);
    printf("Sorted array: ");
    for (i = 0; i < size; i++) printf("%d%s", nums[i], i == size - 1 ? "\n" : " ");
```

```
    occurrence(nums, size);
    return 0;
}
```

Compilation and Execution Commands

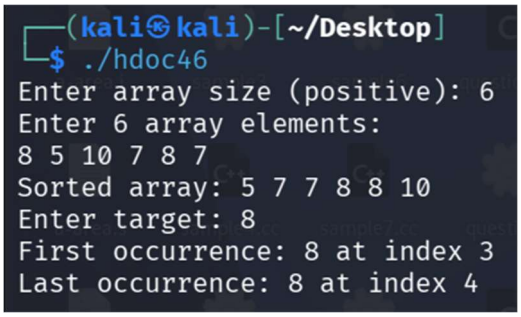
```
gcc hdoc46.c -o hdoc46
```

```
./hdoc46
```

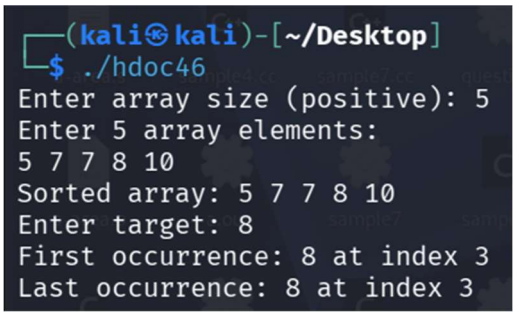
Compilation and Execution



Test Case 1



Test Case 2



Test Case 3

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc46
Enter array size (positive): 5 7 7 8 8 10
Enter 5 array elements:
Sorted array: 7 7 8 8 10
Enter target: 6
-1, -1
```

Test Case 4

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc46
Enter array size (positive): 5
Enter 5 array elements:
4 4 4 4 4
Sorted array: 4 4 4 4 4
Enter target: 4
First occurrence: 4 at index 0
Last occurrence: 4 at index 4
```

Test Case 5

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc46
Enter array size (positive): 1
Enter 1 array elements:
9
Sorted array: 9
Enter target: 9
First occurrence: 9 at index 0
Last occurrence: 9 at index 0
```

Test Case 6

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc46
Enter array size (positive): 5
Enter 5 array elements:
-2 0 -2 3 0
Sorted array: -2 -2 0 0 3
Enter target: -2
First occurrence: -2 at index 0
Last occurrence: -2 at index 1
```

Test Case 7

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc46
Enter array size (positive): 0
Size must be positive.
Enter array size (positive): 3
Enter 3 array elements:
3 1 2
Sorted array: 1 2 3
Enter target: 2
First occurrence: 2 at index 1
Last occurrence: 2 at index 1
```

Test Case 8

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc46
Enter array size (positive): 4
Enter 4 array elements:
9 1 5 3
Sorted array: 1 3 5 9
Enter target: 9
First occurrence: 9 at index 3
Last occurrence: 9 at index 3
```